

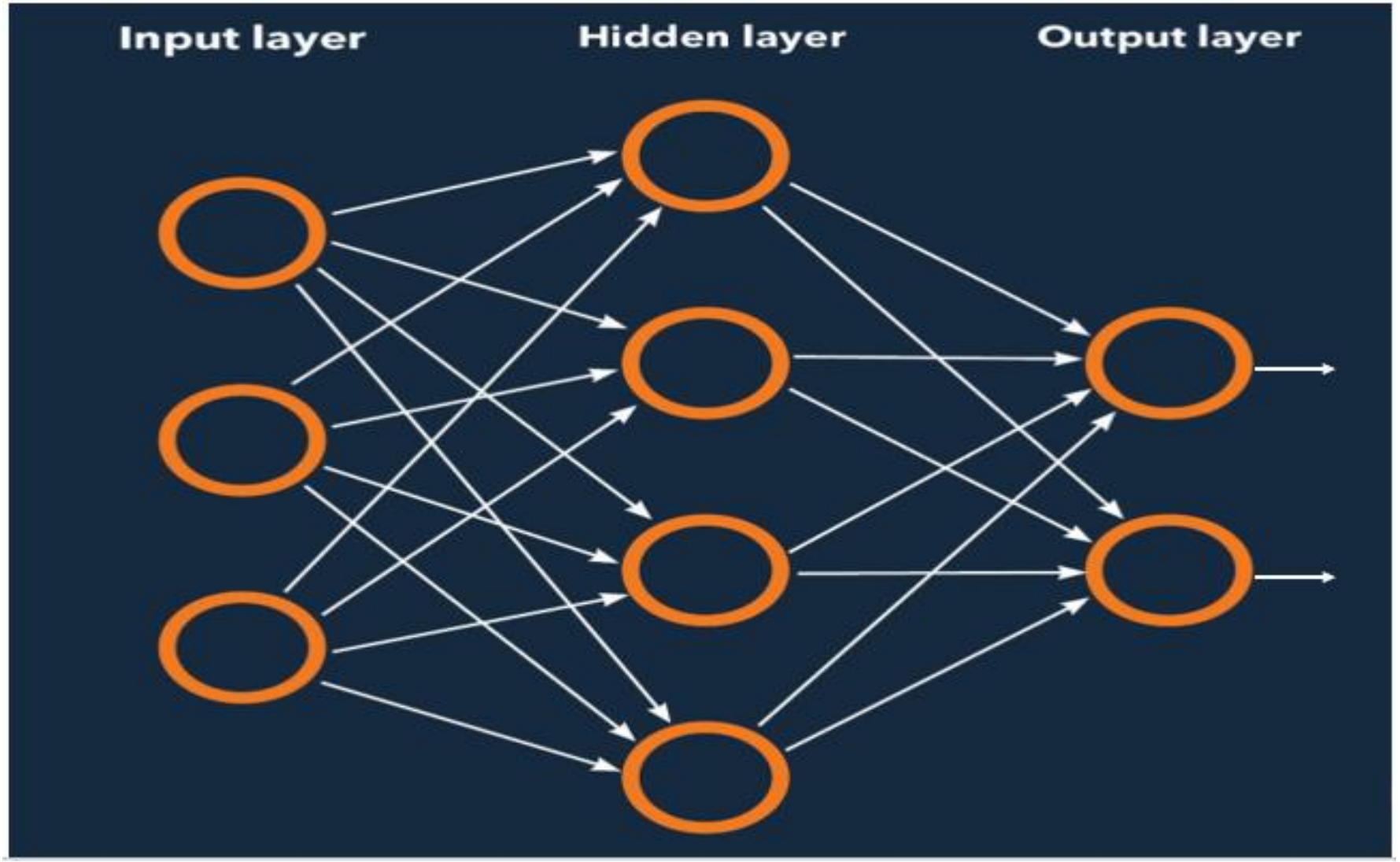
Chapter-4

Neural Network and Deep Learning

Neural Network

- Neural Network is a computational model inspired by the structure and function of biological neural networks in the brain.
- NN consists of interconnected units called neurons, or *nodes*, typically organized in layers(input layer, one or more hidden layers, and an output layer).
- Neurons are basic units that receive input, process it, and produce output.
- Each connection has an associated weight, and neurons apply activation functions to determine output.

Simple Neural Network



Cont'd

❖NNs excel at identifying complex patterns and relationships within data.

❖This makes them powerful tools for tasks like;

- image and speck recognition,
- natural language processing,
- Autonomous vehicles,
- Healthcare diagnostics
- forecasting, and
- control systems where traditional algorithms might struggle.

❖They form the foundation of deep learning.

Characteristics of Neural network

❖Composed of layers of nodes (neurons).

- **Input Layer:** Receives raw data (e.g., pixels in an image, text tokens).
- **Hidden Layers:** Intermediate layers where computation occurs .
- **Output Layer:** Produces the final result (e.g., classification, prediction).

❖Learn from data by adjusting parameters.

- Weight: Parameters that determine the strength of connections between neurons.
- Biases: Offsets added to the weighted sum of inputs to adjust output.

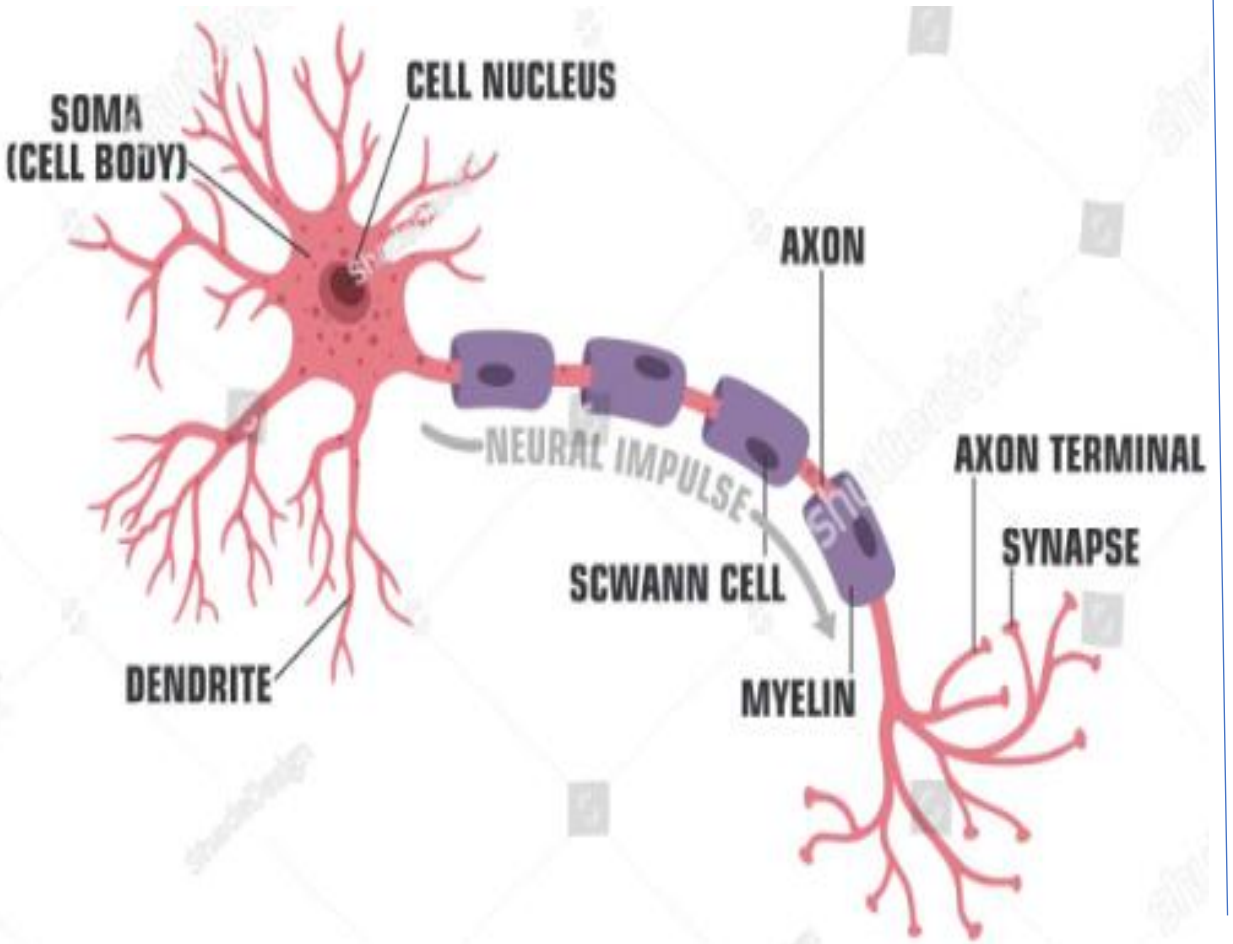
❖Handle complex, nonlinear relationships.

❖Uses Activation Functions

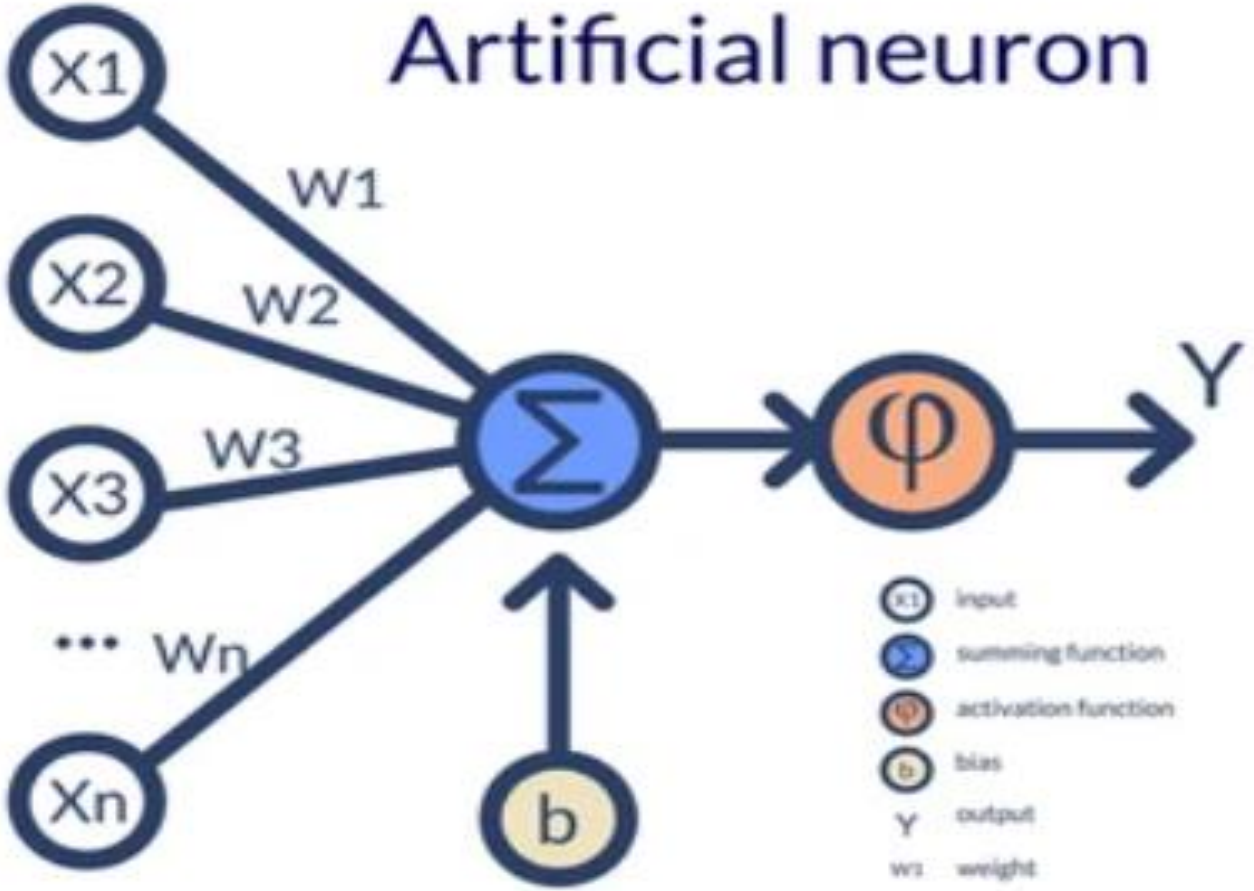
- Introduce non-linearity to model complex relationships.

Biological Inspiration and Perceptron

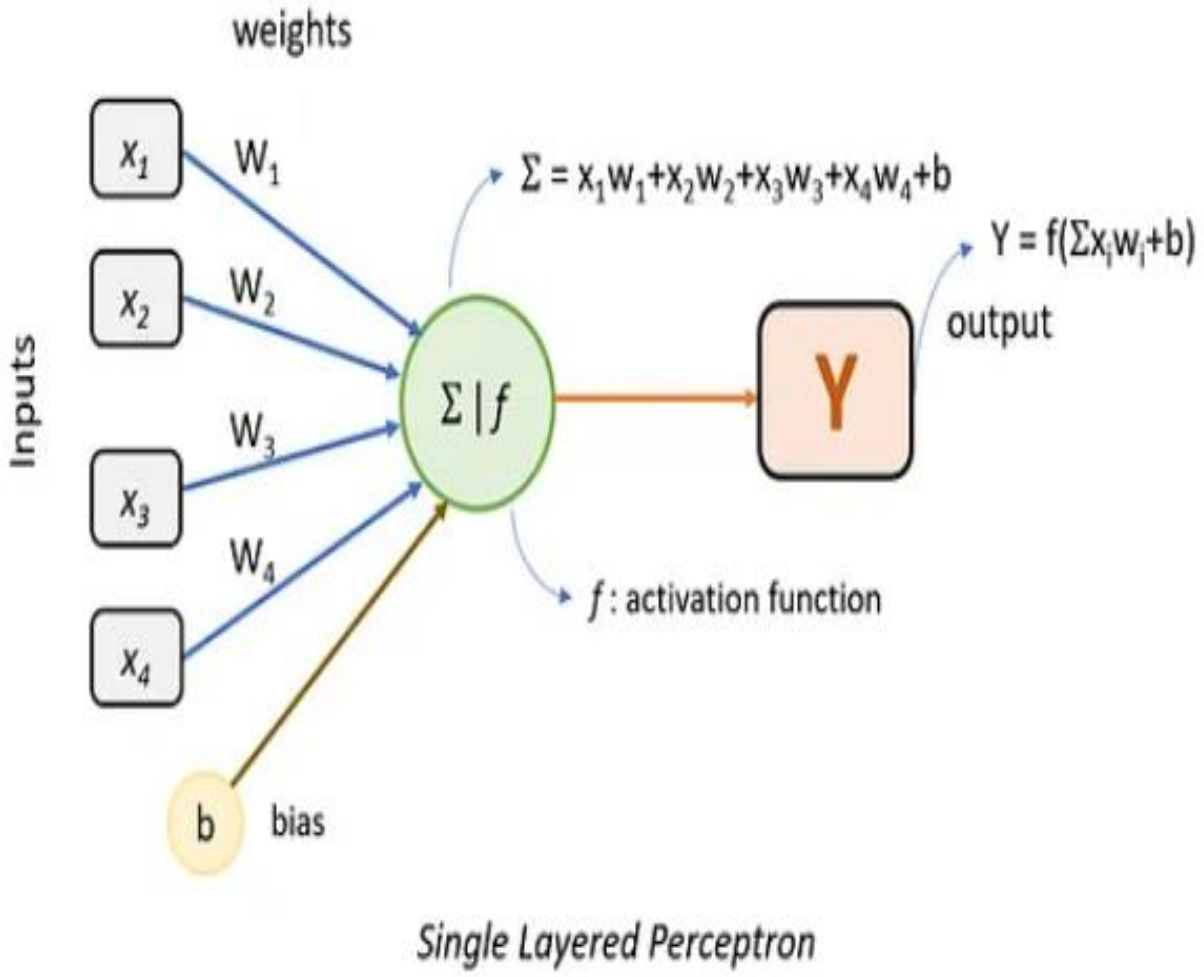
Structure of Biological Neuron



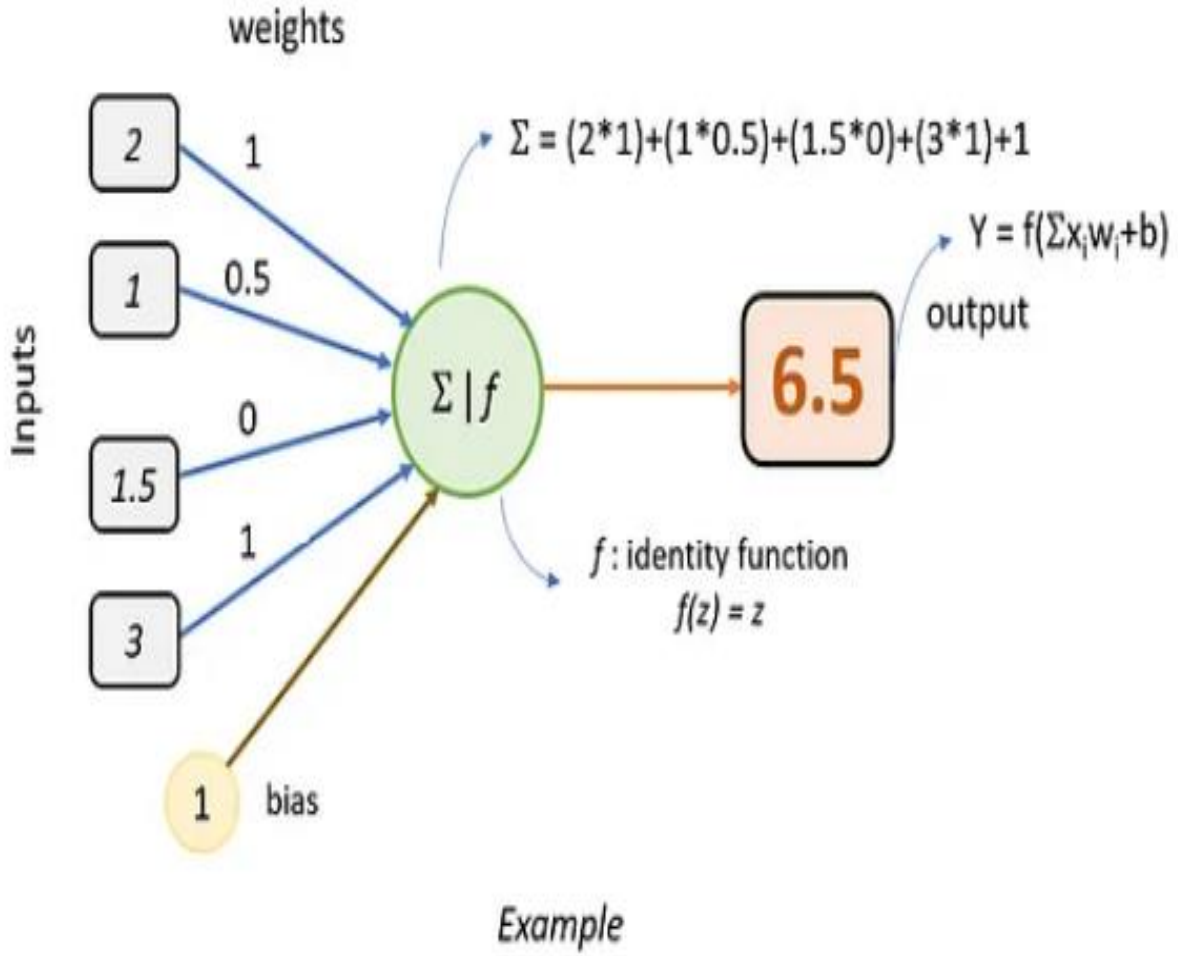
Structure of Artificial Neuron



Example



Single Layered Perceptron

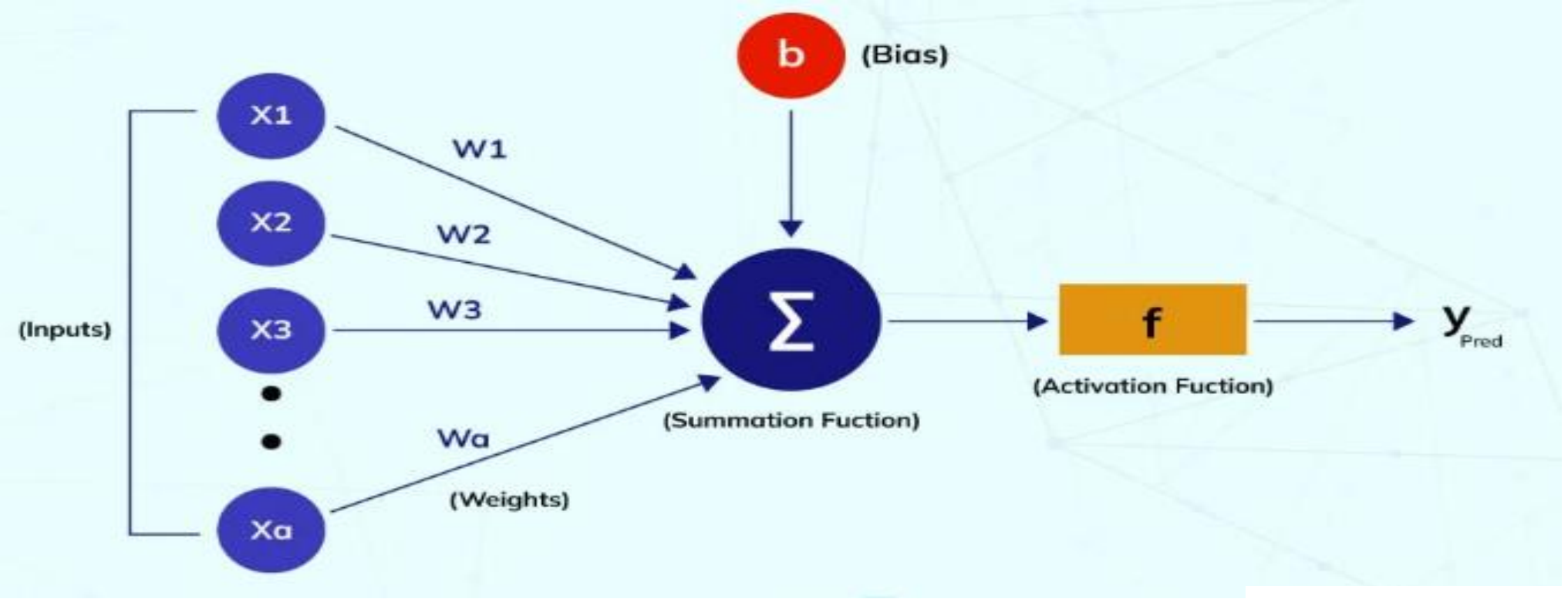


Example

Perceptron

- The Perceptron is one of the simplest type of neural network , introduced by Frank Rosenblatt in 1957.
- It is primarily used for binary classification, usually classifying data into one of two categories, such as 0 or 1.

Perceptron: Building block of neural network



y=$f\left(\sum_{i=1}^{n} wi.xi+b\right)$ where f is the activation function, w are weights, and b=bias(Threshold)

$$y=\begin{cases}1 & \text{if } \sum_{i=1}^{n} w_i x_i + b > 0 \\ 0 & \text{if } \sum_{i=1}^{n} w_i x_i + b \leq 0\end{cases}$$

Cont'd

❖Composed of:

- Inputs (x1,x2,...,xn): Features or data points fed into the perceptron.
- Weights (w1,w2,…,wn). The importance of input is determined by the corresponding weights
- Bias: A bias term (b) is added to shift the decision boundary.
- Activation function: The perceptron uses a **step function** (a simple thresholding function) to determine whether the weighted sum of inputs plus the bias is above or below a certain threshold.
- Output: The output of a perceptron is binary (1 or 0), making it suitable for linearly separable classification problems.

Activation Functions

❖Activation functions introduce non-linearity into the network, enabling it to learn/model complex patterns beyond linear boundaries..

❖ReLU (Rectified Linear Unit)

f(x)=max(0, x)

Output Range: [0, ∞)

❖Sigmoid:

$$f(x) = \frac{1}{1+e^{-x}}$$

Outputs between 0 and 1

Tanh (Hyperbolic Tangent)

$$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

Outputs between -1 and 1

Step function

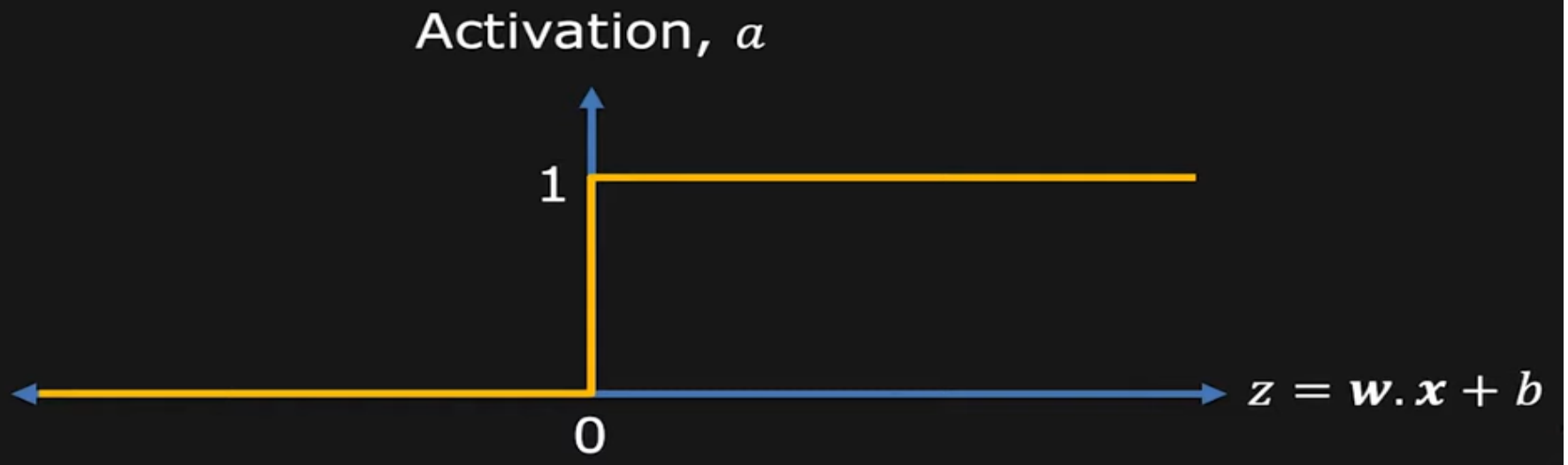
f(z)= {1 if z>0, 0 if z<=0

Outputs either 1 and 0

Step activation Function

Let: $z = \boldsymbol{w}.\boldsymbol{x} + b$

Activation: $a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$



Activation Function for a Perceptron is a Step Function.

Why do we Need Weight and Bias?

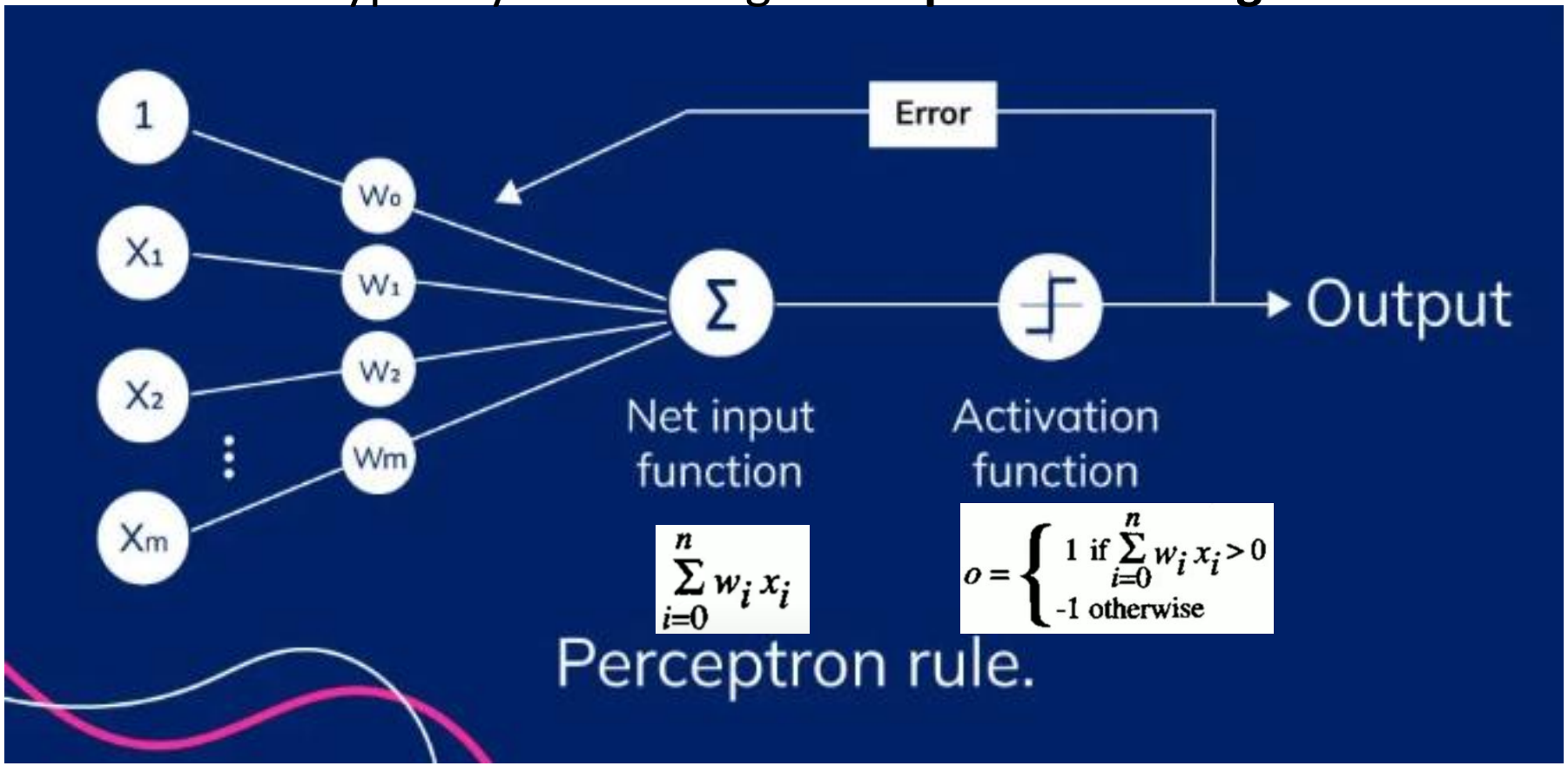
Machine Learning

- Weight and bias are learnable parameters. When the network gets trained, it adjusts both parameters to achieve the desired values and the correct output.
- Weights are used to measure the importance of each feature in predicting output value. Features with values close to zero are said to have lesser significance. These have less importance in the prediction process compared to the features with a larger value. Besides high-weighted features having greater predictive power than low-weighting ones, the weight can also be positive or negative. If the weight of a feature is positive then it has a direct relation with the target value, and if it is negative then it has an inverse relationship with the target value.
- Bias acts like an intercept in a linear equation. Bias is a constant used to adjust the output and help the model to provide the best fit output for the given data.

Training the Perceptron

Training a perceptron involves finding the optimal **weights** (and **bias**) that allow it to correctly classify the input data. This is typically done using **Perceptron Learning Rule.**

Goal: Minimize the errors between the perceptron's output and the actual target labels in the training data



Perceptron Learning Rule (Iterative Process):

1. Initialization: Initialize weights (w) and bias (b) to small random values or zero.
2. For each training example (x, y_{target}):
   - a. Calculate Output: Compute the perceptron's output for a given input.
   - b. Error Calculation: Compare the output with the actual label.
   - c. Update Weights and bias: If the prediction is incorrect ($y_{pred} \neq y_{target}$), Adjust weights using the perceptron learning rule:
     - $w_i(new)=w_i(old)+\Delta wi$,$\Delta w_i=\eta(y_{target}-y_{pred})x_i$ $\quad$ $Error=y_{target}-y_{pred}$
     - $b(new)=b(old)+\Delta b$,$\Delta b=\eta(y_{target}-y_{pred})$
     - Where η is the *learning rate* (a small positive value, e.g., 0.01, 0.1) that controls the step size of the update.

Cont'd

3. Repeat: Continue iterating through the training examples until the perceptron converges (makes no more errors on the training set) or a maximum number of iterations is reached.

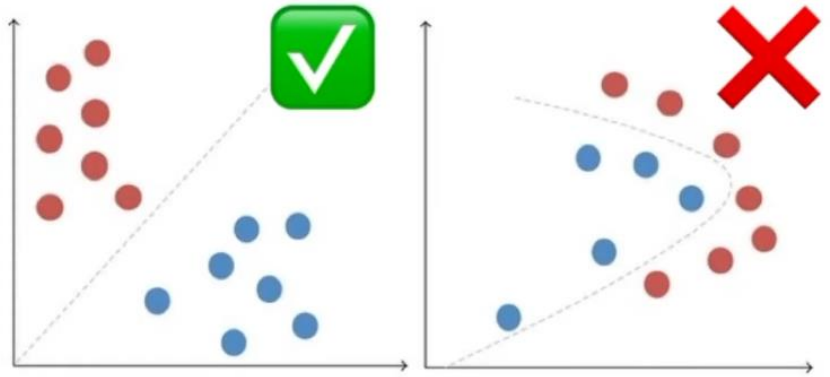
Perceptron Learning Rule



Perceptron_training_rule *(X, η)*
initialize ***w*** *(wi ← an initial (small) random value)*
repeat
  for each training instance ***(x, tx)*** $\in X$
    compute the real output ***ox*** *= Activation(Summation(****w.x****))*
    if ***(tx ≠ ox)***
      for each wi
        $wi \leftarrow wi + \Delta wi$
        $\Delta wi \leftarrow \eta\ (tx - ox)xi$
      end for
    end if
  end for
until all the training instances in X are correctly classified
return ***w***

Limitations of Perceptron:

- Linear separability: The perceptron can only solve problems that are linearly separable (i.e., it can only classify data that can be separated by a straight line or hyperplane). It cannot solve more complex problems like XOR.
- Single-layer model: The original perceptron is a single-layer model and does not have hidden layers, limiting its expressiveness.



Multilayer Perceptron

An MLP is a feedforward neural network with:

- An input layer
- One or more hidden layers
- An output layer

❖Unlike a single-layer perceptron, which can only model linearly separable problems, an MLP can handle complex, non-linear relationships due to its multiple layers and non-linear activation functions.

❖MLP is a supervised learning model that maps input data to output labels using a series of fully connected layers with non-linear activation functions.

❖(MLPs) extend the perceptron by adding multiple hidden layers, enabling them to learn more complex functions.

Structure of MLP:

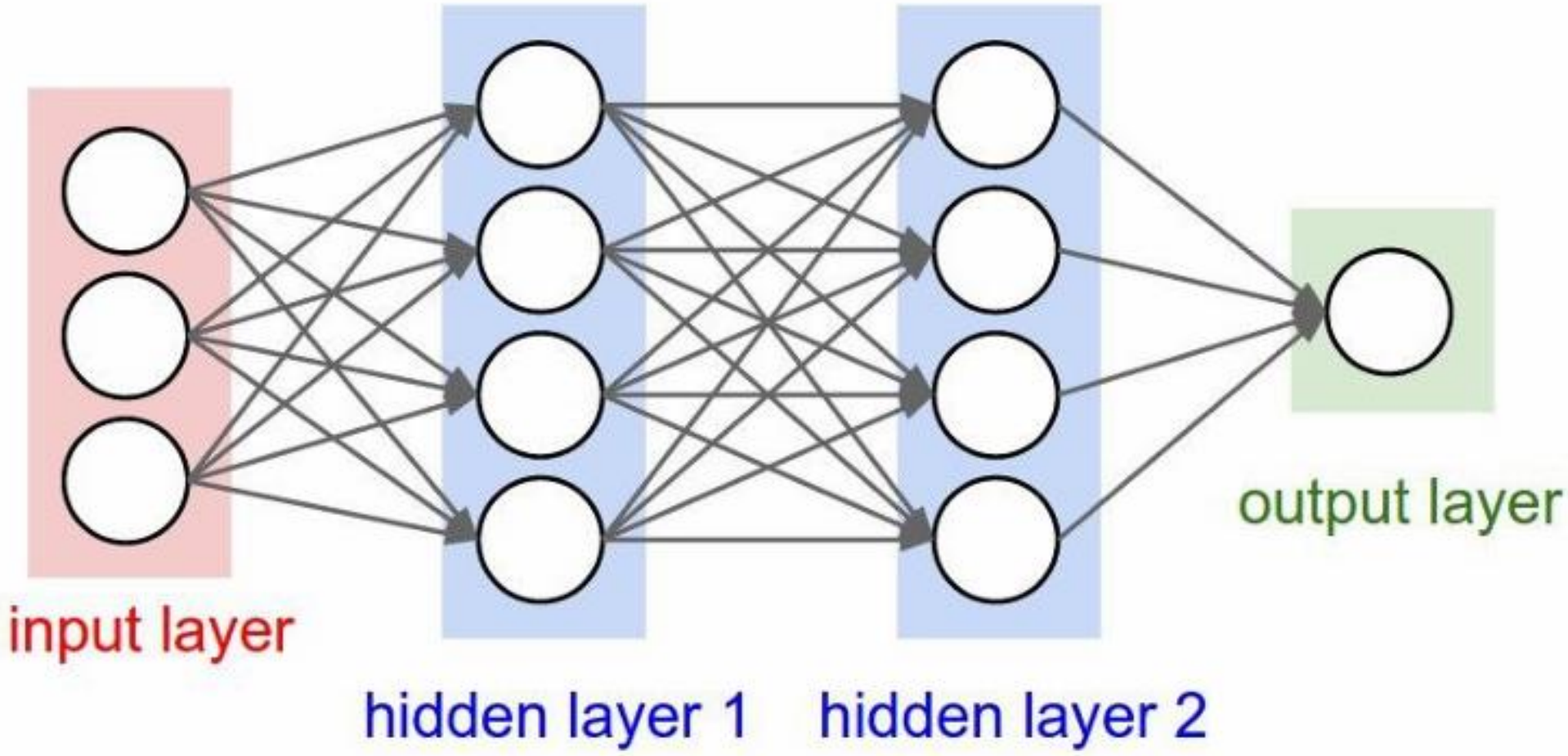
An MLP typically has three types of layers:

❖Input Layer
- Loads data into a model and passes it without any computation
- Number of neurons equals the number of input features.

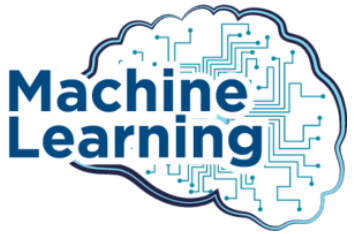
❖Hidden Layers (one or more):
- Intermediate layers between input and output.
- Perform computations to extract patterns or representations.
- These layers use non-linear activation functions (like ReLU, Sigmoid, Tanh) and enable the network to learn complex, non-linear relationships.

❖Output Layer:
- Produces the final prediction (e.g., class probabilities in classification, scalar values in regression).
- Number of neurons depends on the task;
  1. Regression: Single neuron for continuous output.
  2. Binary Classification: One neuron with Sigmoid activation.
  3. Multi-class Classification: One neuron per class with Softmax activation.

Cont'd



Key Features



- Fully Connected (Dense): Every neuron in one layer connects to all neurons in the next layer.
- With sufficient neurons and layers, MLPs can approximate any continuous function
- Supervised learning
- Can model non-linear relationships
- Requires data normalization/standardization
- Typical flow: Input → Hidden Layer(s) → Output
- Works well for structured/tabular data.
- Easy to implement and train on small datasets.

Training MLP

Step-by-Step:

1. Forward Pass – Calculate output using current weights.
2. Loss Calculation – Measure the difference between predicted and actual output. Use a loss function like:
3. Backward Pass (Backpropagation) – Calculate gradients of loss w.r.t. weights.
4. Weight Update and weight– Adjust weights and bias using optimization algorithms - gradient descent.
5. Repeat for multiple epochs.

1. Forward pass

❖In the forward pass(Forward propagation), the input data is passed through the network to compute an output (prediction).

- Initialize the weights: The weights in the network are initialized to small random numbers (e.g., ranging from −1.0 to 1.0, or −0.5 to 0.5). The biases are similarly initialized to small random numbers
- Feed the training sample(input) to the input layer of the network
- Next, compute the net input and output of each unit in the hidden and output layers.

Cont'd

❖Compute network input of a units; The inputs to unit j are outputs from the previous layer. These are multiplied by their corresponding weights in order to form a weighted sum, which is added to the bias associated with unit j.

❖Given a unit j in a hidden or output layer, the net input, Ij;

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

The inputs pass through the input units, unchanged

❖where wij is the weight of the connection from unit i in the previous layer to unit j; Oi is the output of unit i from the previous layer; and θj is the bias of the unit

Cont'd

- The next step is to pass these weighted sums (Ij) through a non-linear activation function (e.g., ReLU, Sigmoid, Tanh); which introduces non-linearity into the model. This is crucial for the network to learn complex patterns.
- Given the net input Ij to unit j, then the output of unit j, is computed as

$$O_j = f(I_j)$$

- where f represents activation function and Ij refers net input to unit j.
- We compute the output values, Oj , for each units in hidden and output layer.
- Activation function used in hidden layer and output layer may be different.

2. Loss Function

The loss function measures the error between the predicted output and the actual target, which will be used during backpropagation.

Mean Squared Error (MSE);

$$\text{Loss} = \frac{1}{2}(y_{\text{true}} - y_{\text{pred}})^2$$

❖Example for Cross-Entropy Loss

$$\text{Loss} = -\sum_i y_i \log(\hat{y}_i)$$

ytrue: Actual label
ypred: Network prediction

Loss function, Chain rule and Memorization:

- In SGD, we pass one input at a time. It passes through the network of neurons(each neuron with an activation function) and produces an output. We calculate the loss by comparing the predicted value(output) with actual value.
- The loss function is chosen based on the problem we are solving (regression, binary or multi class classification).
- Then we calculate the derivative using chain rule. After calculating a derivative it is stored in memory so it can be used again. This technique is called Memorization. This helps in faster calculation as it helps in avoiding calculating same derivative again.

3. Backpropagation of associated errors

- To make the network learn(allow a network to capture a relationships between inputs and outputs), we use back-propagation, a technique that adjusts the **weights** and **bias** based on the **error** between the predicted and actual output.
- This adjustment is controlled by **gradient descent or other algorithms**. Through this iterative process, the network improves its ability to make accurate predictions, ultimately enabling it to solve complex problems.
- The main goal is to minimize the difference(loss) between predicted output and actual output.

3. Backpropagation of associated error

❖Backpropagation computes the gradient (partial derivative) of the loss with respect to each weight in the network using the chain rule.

$$\frac{\partial \text{Loss}}{\partial w} = \frac{\partial \text{Loss}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Where:

- $\frac{\partial \text{Loss}}{\partial a}$: How loss changes with activation
- $\frac{\partial a}{\partial z}$: Derivative of activation function
- $\frac{\partial z}{\partial w} = x$: Input to the neuron

This is computed layer by layer, starting from the output layer back to the input.

4. Weight Update (Using Gradient Descent)

❖After computing the gradients, weights are updated to reduce the error.

$$w := w - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

- η: Learning rate (small positive number)
- $\frac{\partial \text{Loss}}{\partial w}$: Gradient of the loss with respect to weight

This is done for all weights in the network.

5. Repeat for Multiple Epochs

An epoch means one complete pass of the training dataset through the network.

❖Repeat steps 1 to 4 for multiple epochs until:

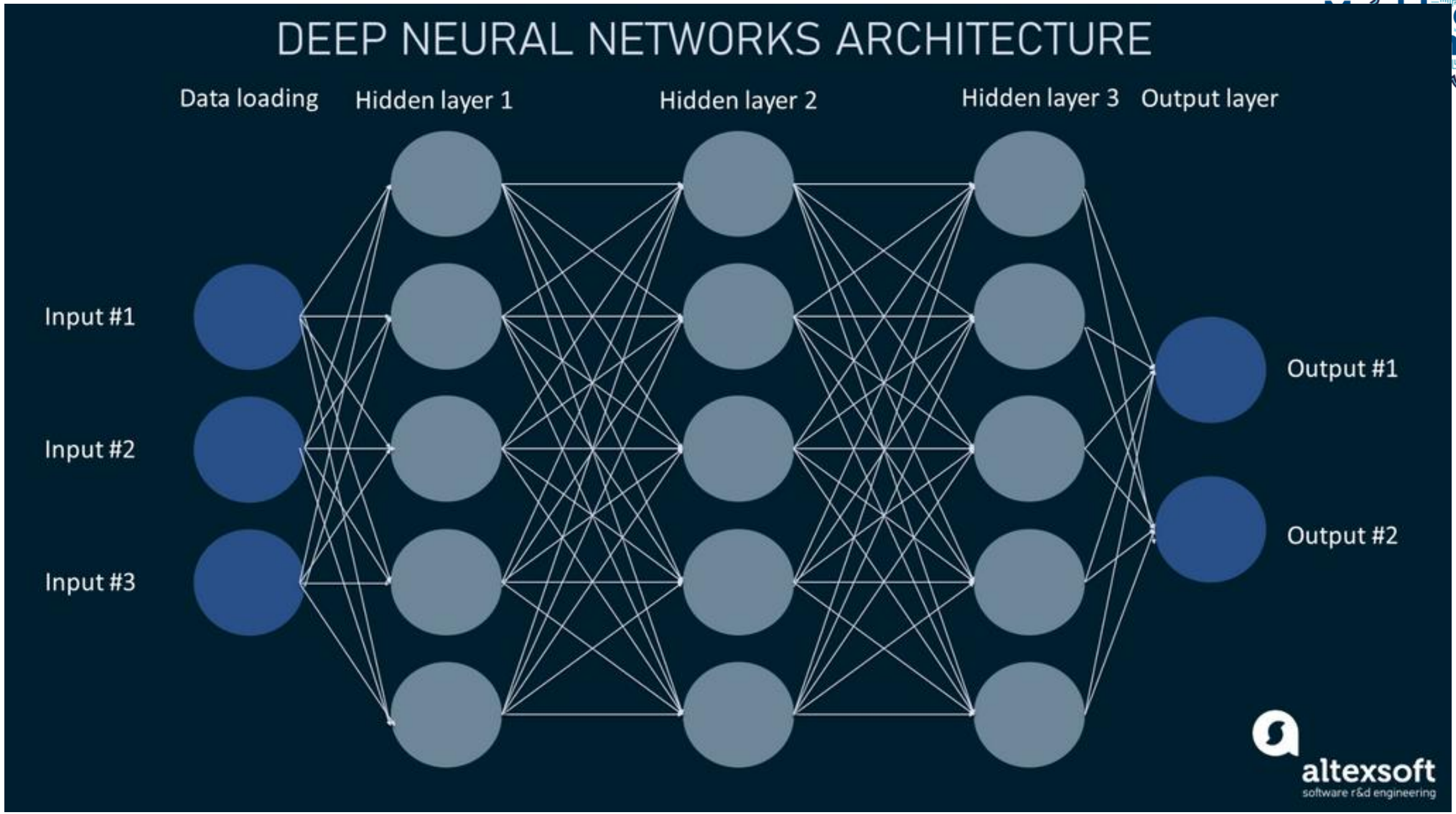
- ✓Loss becomes sufficiently low
- ✓Or accuracy stops improving
- ✓Or maximum number of epochs is reached

Tuning the Network Size

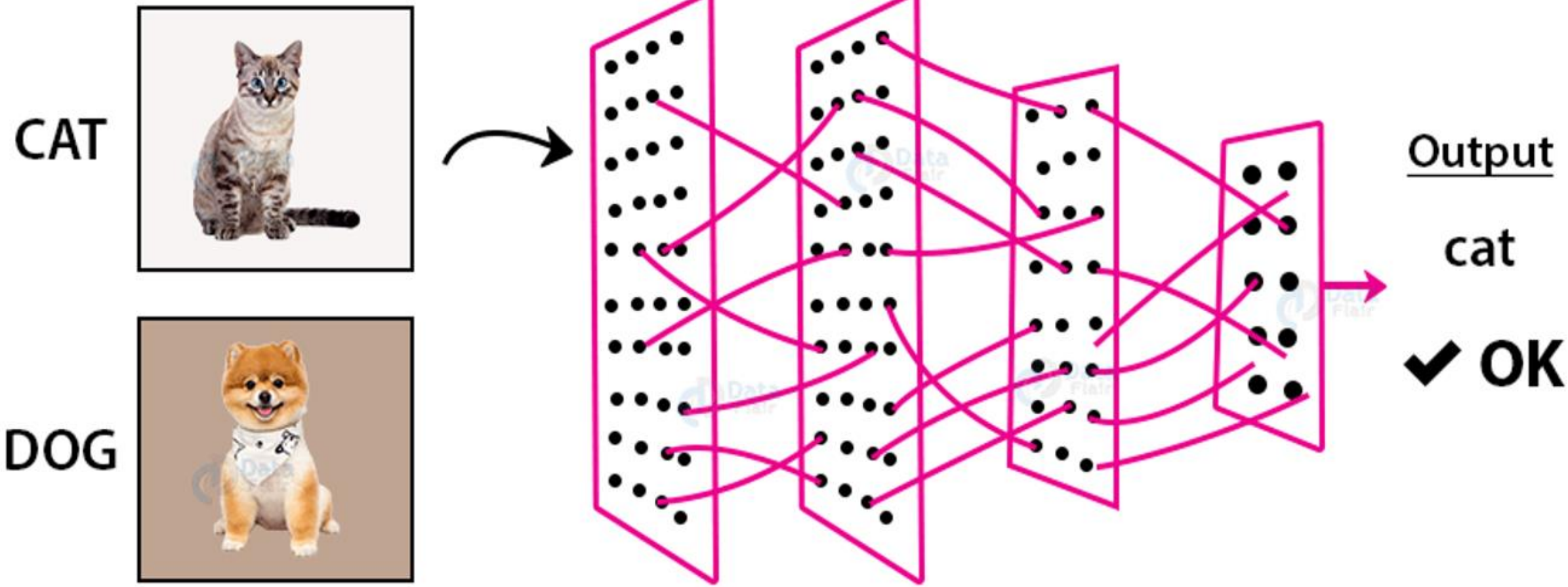
- Number of Layers: More layers allow learning of more complex features.
- Number of Neurons: Too few may underfit, too many may overfit.
- Heuristics:
  - Start small and increase complexity as needed.
  - Use validation performance to guide tuning.
  - Regularization (L1/L2, dropout) helps control overfitting.

Deep Neural Networks (DNNs)

- Deep learning is a subset of machine learning that uses multilayered neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain.
- It is particularly powerful for tasks involving unstructured data such as images, audio, video, and natural language.


DEEP NEURAL NETWORKS ARCHITECTURE
Data loading
Hidden layer 1
Hidden layer 2
Hidden layer 3
Output layer
Input #1
Input #2
Input #3
Output #1
Output #2
altexsoft
software r&d engineering

Cont'd



Popular Architectures:

I. **Convolutional Neural Networks (CNNs)**: Excel at image/video tasks (e.g., object detection).

II. **Recurrent Neural Networks (RNNs)**: Handle sequential data (e.g., text, time series).

III. **Transformers**: Dominant in NLP (e.g., BERT, GPT) via self-attention mechanisms.

IV. **Generative Models**: GANs (generate synthetic data) and VAEs (learn data distributions).

Reading Assignment

I. Convolutional Neural Networks (CNNs)

II. Recurrent Neural Networks (RNNs)

III. Transformers

IV. Generative Models

Case Study in Neural Network

Problem: Handwritten Digit Recognition (MNIST dataset)

Approach:

- Input: 28x28 pixel grayscale images
- Preprocessing: Normalize input data
- Model:
  - Input layer: 784 nodes
  - Hidden layer: 128 ReLU units
  - Output layer: 10 softmax nodes (digits 0-9)
- Loss function: Cross-entropy
- Optimizer: Stochastic Gradient Descent (SGD)

Outcome:

- High accuracy (~98%). Demonstrates the power of MLPs in pattern recognition